



NEXUS-1

PROJECT ROADMAP

From demonstrator to an industrial-grade platform

Staged plan with work-hour estimates · .NET / Angular / SignalR / Kafka

Author: Grigorios Agathangelidis

Electrical & Software Engineer

Build signature NX1-094C79E0A366D34F

June 2026 · Planning document

ESTIMATE, NOT A QUOTE

Hours are planning estimates for senior engineers and depend on team, seniority and final scope.
Scope = Target A (industrial platform). The regulated-deployment path is a separate, non-engineering track.

How to read this roadmap

This plan takes NEXUS-1 from its current state — a complete, client-side **demonstrator** (about 25 screens, the point-kinetics engine, cross-module event behaviour, but no backend, persistence, auth, real-time transport, tests or infrastructure) — to an **industrial-grade platform** on the stated stack: a .NET / C# back end exposed through REST, an Angular front end, SignalR for real-time push, and Kafka for queueing, deployed as a set of **microservices**. Most of the work is not re-skinning the tabs; it is building the distributed system the tabs sit on top of.

Scope assumed (state your real answers and these numbers shift):

- **Target A — industrial platform.** Productionised monitoring / analytics / digital-twin / decision-support layers, validated against simulation and historian data and integration-ready. NOT the regulated, plant-connected path (covered separately, not estimated as engineering).
- **Architecture — microservices.** Independently deployable services aligned to the bounded contexts on the next page, communicating asynchronously over Kafka and synchronously through an API gateway, with database-per-service ownership. This buys independent scaling and deployability at the cost of extra platform, integration-testing and coordination effort (reflected in P0, P1, P5 and P7).
- **Team.** A small team of senior engineers (backend / frontend / platform-DevOps, with data/ML and part-time architecture). Estimates are senior-engineer hours.
- **'Complete'.** All current tabs taken to production grade, but sequenced vertical-slice-first (see milestone M1) so value and de-risking come early.

How the hours work:

- Each task carries a **Low** and **High** estimate of focused engineering effort (design + build + test + review).
- The **Planning** figure per phase is the midpoint plus ~15% contingency — use this for budgeting.
- A separate line covers **program management & technical leadership** spread across all phases.
- Calendar time is derived assuming ~6 productive hours per engineer per day (~126 / month). Pure division understates duration: coordination, dependencies and ramp-up add overhead.
- Excluded from hours: hardware/licences, third-party services, and the regulatory track.

Architecture: microservices decomposition

The platform decomposes into independently deployable services aligned to the bounded contexts already visible in the application menu. Each owns its data, publishes domain events to Kafka, and is reached synchronously only through the API gateway. Cross-cutting concerns (gateway, mesh, identity, observability) are shared platform, not per service.

Service	Responsibility	Backs (tabs)
Identity & Access	AuthN/Z, RBAC, sessions, tokens	(all — security)
API Gateway / BFF	Edge routing, aggregation, auth enforcement	(all)
Realtime / Notification	SignalR fan-out, presence, push	(all live screens)
Simulation / Digital Twin	Physics, point-kinetics, fleet state, twin	Overview, Reactor, Kinetics, 3D, Twin
Telemetry & Historian	Ingest, time-series store, queries	Trends, Power, Coolant, Steam
Command	Rod / SCRAM control path, validation	Control Rods
Alarm	Alarm rules, dedup, ack, escalation	Alarms & Events
Audit	Hash-chained tamper-evident action log	Compliance / Audit
Asset & Health	Component registry, live health, aging	Component Registry, Aging
Inspection	NDT records and inspection workflow	Rod Inspection
Personnel	Staffing, rosters, coverage rules	Personnel
Robotics	Robot / vehicle fleet, mission readiness	Robotics & Vehicles
Zone Access	Live presence, permissions matrix	Zone Access
Incident & Root-cause	Event store, timeline, causal analysis	Incident, Root Cause Graph
Analytics / AI	Predictive maintenance, dependency / causal	AI Diagnostics, Dependencies
RL Optimiser	Twin-only advisory optimisation sandbox	Optimization (RL)
Protocol Gateways (P6)	OPC-UA / MQTT / Modbus, read-only ingest	(integration, optional)

Start coarse-grained: several of these can begin life merged (e.g. Asset & Health with Inspection, Analytics with RL) and be split later as load and team structure justify it. Splitting too early is the most common microservices mistake and inflates P0/P7.

Executive summary

Phase	Stage	Low	High	Planning
P0	Foundations & Architecture	345	595	540
P1	Backend Core, Simulation Port & Real-time Spine	730	1,200	1,110
P2	Angular Frontend Rebuild	630	1,090	990
P3	Non-simulation Domain Services	680	1,130	1,040
P4	AI / Analytics Services	390	670	610
P5	Security & OT Hardening	380	660	600
P7	Production Readiness & QA	630	1,110	1,000
PM	Program management & technical leadership (all phases)	520	820	770
CORE PLATFORM TOTAL (Target A)		4,305	7,275	6,660
P6	Protocol integration (optional, shadow mode)	400	720	640
TOTAL incl. optional protocol integration		4,705	7,995	7,300

All figures in engineering work-hours. Planning = midpoint + ~15% contingency.

Indicative calendar (Target A core, planning hours)

Team	Composition	Naive	Realistic*
Solo	1 senior full-stack	~53 mo	~3.5-4 yr — not advised (breadth, no parallelism)
Small (3)	backend / frontend / platform-DevOps	~18 mo	~15-18 months to a hardened v1
Balanced (5-6)	2 BE, 2 FE, 1 SRE, 1 data/ML, p/t architect+QA	~11 mo	~10-13 months to a strong v1

*Realistic figures add coordination overhead, inter-phase dependencies and ramp-up; some phases (P5, P7) run in parallel with P3/P4. Protocol integration (P6) adds ~4-6 months, parallelisable.

Milestones & sequencing

Do not build all 25 tabs at once. The strongest sequencing is one **vertical slice through the entire stack first**, then fan out the remaining domains (which mostly repeat established patterns).

Milestone	Content	Approx. hours
M1 — Vertical slice	End-to-end: P0 essentials + minimal P1 (sim, API, SignalR, Kafka, historian) + P2 (shell, Overview, Fleet, live Reactor) + Alarms + Audit + auth. Proves the hard parts.	~750-950
M2 — Simulation tier	Remaining P1 + P2: full reactor, kinetics, 3D, steam/coolant, trends on real data.	~1,300-1,700
M3 — Domains complete	All P3 services and screens backed by real data and workflow.	~700-1,150
M4 — Analytics + security	P4 analytics tier and P5 security hardening (largely parallel).	~770-1,330
M5 — Production v1	P7 complete: tested, observable, resilient, deployed on Kubernetes.	~540-940
M6 — Integration (opt.)	P6 read-only shadow-mode protocol gateways.	~400-720

M1 is the most important decision in this plan: ~3-4 months for a small team that yields a demonstrably industrial end-to-end system before a year of spend, and validates every estimate below.

Detailed stages (work breakdown)

Foundations & Architecture

P0

Stand up the skeleton everything else hangs from: architecture, environments, pipelines and the auth model. No business features yet — this phase exists to de-risk the build.

ID	Task	Low	High	Notes
P0.1	Microservices architecture, ADRs, service decomposition	55	90	Service boundaries, ownership, topic design
P0.2	.NET solution scaffolding + shared libraries	20	35	Projects, shared kernel
P0.3	Angular workspace + design-system scaffold	25	40	Port the existing CSS tokens
P0.4	Local dev environment (docker-compose)	30	50	Kafka, Postgres, TimescaleDB, IdP
P0.5	CI/CD pipeline skeleton (per-service)	35	60	Build / test / lint / containerize each service
P0.6	IaC skeleton (Terraform / Helm)	30	55	Baseline cluster + environments
P0.7	Identity / auth selection + RBAC model	25	45	Keycloak / Duende / Entra
P0.8	Event & telemetry contracts + schema registry	30	50	Avro / Protobuf, versioning
P0.9	Real-time throughput spike (SignalR + Kafka)	25	45	Prove the spine early
P0.10	Standards, repo, branching, board setup	15	25	Conventions & templates
P0.11	Service template / golden-path scaffold	25	45	Health, logging, tracing, build baked in
P0.12	API gateway + service discovery + central config	30	55	YARP / Ocelot, K8s DNS / Consul
Phase subtotal		345	595	Planning ~540 h

- › Highest leverage of the whole programme — a weak P0 multiplies cost everywhere downstream.
- › Output: a running empty skeleton that builds, deploys and authenticates.

Backend Core, Simulation Port & Real-time Spine

P1

Build the heart of the platform: the server-side simulation/twin, the API, and the SignalR + Kafka + historian pipeline that streams live state to clients.

ID	Task	Low	High	Notes
P1.1	Core domain model + persistence (EF Core / Postgres)	50	80	Units, components, events
P1.2	Simulation/Twin service — fleet model & master tick	70	110	Port the multi-unit simulation
P1.3	Point-kinetics engine port + unit tests	90	140	HIGH RISK — 6 delayed groups, feedbacks
P1.4	Thermal-hydraulics, secondary & grid models	50	80	Coolant / steam / power
P1.5	Deterministic sim test harness (golden-master)	40	70	Reproducibility & regression
P1.6	Edge API gateway + BFF + per-service contracts	60	95	Versioned, OpenAPI, aggregation
P1.7	SignalR hubs + backpressure	50	80	Telemetry / presence streams
P1.8	Kafka integration (topics, registry, idempotency)	70	110	Producers / consumers
P1.9	TimescaleDB historian write path + retention	50	80	Downsampling, continuous aggregates
P1.10	Command path (rod / SCRAM) + audit emission	40	70	Consequential actions
P1.11	Read models, caching, health checks	30	50	Query side

ID	Task	Low	High	Notes
P1.12	API error model, configuration, feature flags	40	80	Cross-cutting
P1.13	Inter-service comms + consistency (outbox / saga)	60	100	Async events + sync gRPC / REST
P1.14	Per-service data ownership + migrations	30	55	DB-per-service, no shared schema
Phase subtotal		730	1,200	Planning ~1,110 h

- › The point-kinetics port (P1.3) is the single biggest technical risk — protect its schedule and test it hard.
- › Output: live, persisted plant state streaming to a client over SignalR.

Angular Frontend Rebuild

P2

Re-implement the console as a proper Angular application driven by the API and the real-time stream, mirroring the existing menu, with the visualization-heavy screens rebuilt.

ID	Task	Low	High	Notes
P2.1	App shell, routing (menu mirror), theming	60	100	Design-system port
P2.2	Auth (OIDC), route guards, RBAC-aware UI	40	70	
P2.3	Generated API client + SignalR client / reconnect	40	70	
P2.4	State management + real-time store	50	90	Signals / NgRx
P2.5	Simulation screens: Overview / Fleet / Power	60	100	
P2.6	Reactor screens: Core / Rods / Neutronics / Steam / Coolant	80	130	Interactive controls
P2.7	Kinetics screen (charts + controls)	40	70	
P2.8	3D screens (Reactor 3D + Plant 3D)	90	150	three.js, performance
P2.9	Trends / History (time-series)	40	70	
P2.10	Shared component library, a11y, responsive	50	90	
P2.11	i18n scaffold (EN / EL)	20	40	
P2.12	Frontend tests (unit / component / e2e)	60	110	Cypress / Playwright
Phase subtotal		630	1,090	Planning ~990 h

- › Output: the simulation tier of the console, production-grade, on real data.

Non-simulation Domain Services

P3

Turn the record-keeping and workflow tabs into real services with persistence and process: alarms, the tamper-evident audit, the asset/inspection/personnel/robotics/access domains, and incident + root-cause analysis.

ID	Task	Low	High	Notes
P3.1	Alarm engine + ack / escalation workflow + UI	90	140	Dedup, severity, routing
P3.2	Tamper-evident audit (hash-chained, append-only) + UI	70	110	Real crypto + verification
P3.3	Asset / component registry + live health + UI	70	110	Per-unit equipment
P3.4	Aging & degradation service + UI	50	80	
P3.5	Inspection / NDT records + workflow + UI	60	100	
P3.6	Personnel / staffing + coverage rules + UI	60	100	
P3.7	Robotics & vehicles + mission readiness + UI	60	100	
P3.8	Zone access (presence + permissions matrix) + UI	50	90	

ID	Task	Low	High	Notes
P3.9	Incident analysis (event store + timeline) + UI	50	90	
P3.10	Root-cause graph service + ranking + UI	70	120	
P3.11	Domain events on Kafka + projections	50	90	
Phase subtotal		680	1,130	Planning ~1,040 h

- › The alarm engine and the cryptographic audit log are larger than they look — budget accordingly.
- › Output: every non-simulation tab backed by a real data model and workflow.

AI / Analytics Services

P4

Build the analytics tier on top of the historian: predictive maintenance, the dependency/causal model, and the RL optimiser kept strictly as a twin-only advisory sandbox.

ID	Task	Low	High	Notes
P4.1	Feature pipeline off the historian	60	110	TimescaleDB -> features
P4.2	Predictive-maintenance baselines + model serving	90	150	Statistical first, then ML
P4.3	Dependency / causal service	60	100	Graph compute, weights
P4.4	RL optimiser (twin-only, advisory)	90	150	Sandbox + readable policy
P4.5	Model registry, evaluation, drift monitoring	50	90	
P4.6	Advisory UI surfacing + guardrail labelling	40	70	
Phase subtotal		390	670	Planning ~610 h

- › Realism here is bound by data — start transparent/heuristic, become ML once historian data exists.
- › Advisory only: never wired to control or safety actuation.

Security & OT Hardening

P5

Bring the platform to critical-infrastructure security posture: identity, service-to-service trust, secrets, segmentation, supply-chain scanning and an independent penetration test.

ID	Task	Low	High	Notes
P5.1	AuthN/Z hardening + RBAC enforcement tests	60	100	
P5.2	mTLS / service identity, secrets (KMS), HSM	70	120	
P5.3	Segmentation / zero-trust, gateway hardening	50	90	
P5.4	Security audit coverage + SIEM hooks	40	70	
P5.5	Threat model + SCA / SAST / DAST in CI	50	90	
P5.6	External penetration test + remediation	80	140	Third-party engagement
P5.7	OT security policy pack + documentation	30	50	
Phase subtotal		380	660	Planning ~600 h

- › Several items run in parallel with P3/P4; the pen test is best near feature-complete.

Production Readiness & QA

P7

Make it operable and trustworthy in production: the full test pyramid, load/soak/chaos testing of the real-time pipeline, observability, HA/DR and a Kubernetes production deployment. Runs largely in parallel with P3-P5.

ID	Task	Low	High	Notes
P7.1	Test strategy + pyramid (unit / integration / contract)	90	150	
P7.2	Load & soak testing (pipeline / Kafka / SignalR fan-out)	70	120	
P7.3	Chaos / resilience + failover drills	50	90	
P7.4	Observability (OpenTelemetry, dashboards, alerting)	70	120	
P7.5	HA / redundancy + backpressure tuning	60	100	
P7.6	Kubernetes production deploy (Helm, autoscaling)	80	140	
P7.7	DR + backup / restore + air-gapped support	60	110	
P7.8	Documentation, runbooks, ops, API reference	60	110	
P7.9	Service mesh + resilience patterns	50	90	Circuit breakers, retries, bulkheads, timeouts
P7.10	Consumer-driven contract testing (Pact)	40	80	Cross-service contracts enforced in CI
Phase subtotal		630	1,110	Planning ~1,000 h

› Output: a deployable, observable, resilient v1 of the platform.

Protocol Integration — optional, shadow mode

P6

Read-only integration with plant data through isolated gateway services. Optional and deferrable; only meaningful once a data source is available.

ID	Task	Low	High	Notes
P6.1	Gateway architecture + isolation	40	70	
P6.2	OPC-UA gateway	80	140	
P6.3	MQTT gateway	50	90	
P6.4	Modbus TCP gateway	50	90	
P6.5	Normalization / validation to canonical model	60	110	
P6.6	Read-only twin sync to live data (divergence)	70	130	Shadow mode
P6.7	Conformance / integration test rig	50	90	
Phase subtotal		400	720	Planning ~640 h

› Strictly read-only; no command path to plant equipment.

Connecting the platform to, and having it trusted by, a real nuclear plant is a different undertaking from the engineering above. It is largely organisational and specialist-led, and it is **not estimated here in engineering hours** because doing so honestly requires a nuclear QA organisation rather than a software team. Indicative activities and an order-of-magnitude duration:

- Formal verification & validation of the simulation/twin models against qualified references.
- Requirements traceability and a documented safety / security case.
- Alignment with the relevant standards (e.g. IEC 61513 system level, IEC 60880 software, IEC 62645 cyber-security).
- Independent verification and assessment by a separate competent party.
- Regulator engagement and licensing-basis documentation.
- Even then, the platform remains in the monitoring / analytics / supervisory layer; certified reactor-protection functions stay hardware-isolated, deterministic and outside this system.

Indicative duration: 12-36+ months, specialist-led, partly in parallel with later engineering phases. Treat as a programme, not a sprint.

Assumptions, Caveats & Authorship

- These are planning estimates, not a fixed-price quotation. Actuals depend on team seniority, the final agreed scope, and the answers to the open scoping questions (team size/mix, target depth, and definition of 'complete').
- The point-kinetics port (P1.3) is the dominant technical risk; the alarm engine and cryptographic audit (P3.1, P3.2) and the real-time pipeline (P0.9, P1.7, P1.8, P7.2) are the next most uncertain.
- Kafka with a single simulation producer is future-proofing for multi-source ingestion, not a present functional need; it adds operational weight early but suits the architecture and integration goals.
- Microservices add platform, contract-testing and coordination overhead (P0.11-12, P1.13-14, P7.9-10, and the program-management line). Begin with coarse-grained services and split only when load or team topology demands it; over-decomposition is the main way this estimate would overrun.
- Re-validate every phase estimate at the end of Milestone M1 — the vertical slice converts these ranges into measured velocity.

This roadmap and the NEXUS-1 console are the original work of Grigorios Agathangelidis. Build signature NX1-094C79E0A366D34F.